Quicksort: Partition, Then Recurse

Programming and Data Structures — Week 6, Lecture 2

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to implement quicksort entirely from scratch, including the Lomuto partition scheme; trace the partitioning process completely, understanding precisely how the pivot element ends up in its correct final sorted position after one partition pass; explain why quicksort's average-case running time is $\Theta(n \log n)$ while its worst case is $\Theta(n^2)$, using the same recursion-tree reasoning developed for merge sort; and explain why quicksort sorts in place, using O(log n) extra space for recursion rather than merge sort's O(n) extra space for merging.

Outline

Today covers a pivot-and-partition analogy; the partitioning process, which is the true engine driving quicksort, traced completely; quicksort's own recursive structure; a complete trace of quicksort's recursion on an 8-element input; the sharp contrast between its average-case and worst-case recursion trees; why the worst case arises and how pivot selection addresses it; a direct space-cost comparison against merge sort; an in-class quiz; and a summary.

The pivot-and-partition analogy

Picture picking a single card from an unsorted hand — this is the pivot. Every other card is then rearranged around it: cards smaller than the pivot move to its left, cards larger move to its right. After this single rearrangement, the pivot sits in its exact final sorted position within the whole array — no future step of the algorithm will ever need to move it again. The two groups on either side, still themselves unsorted, are then each processed by repeating this exact same idea independently — the recursive heart of quicksort.

Partitioning: the Lomuto scheme, in code

```
def partition(arr, low, high):
    pivot = arr[high]        # choose the LAST element as the pivot
    i = low - 1               # boundary of the "smaller than pivot" region
    for j in range(low, high):
        if arr[j] < pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[high] = arr[high], arr[i + 1]   # place pivot in its final spot
    return i + 1               # the pivot's final index
```

The Lomuto partition scheme chooses the last element in the current range as the pivot. The index `i` tracks the right boundary of the region already confirmed to be smaller than the pivot. Scanning `j` from `low` to `high−1`, whenever an element smaller than the pivot is found, `i` advances by one and that element is swapped into the newly expanded "smaller" region. After the scan completes, one final swap places the pivot itself into position `i+1`, exactly at the boundary between everything smaller (to its left) and everything larger (to its right) — this is the pivot's genuine final sorted position.

Tracing partition completely on `[8, 3, 5, 1, 9, 2]`

```
pivot = arr[5] = 2.  i = -1
j=0: arr[0]=8, not <2 → no change
j=1: arr[1]=3, not <2 → no change
j=2: arr[2]=5, not <2 → no change
j=3: arr[3]=1, IS <2 → i=0, swap arr[0],arr[3] → [1,3,5,8,9,2]
j=4: arr[4]=9, not <2 → no change

Final swap: arr[i+1]=arr[1], arr[high]=arr[5] → swap → [1,2,5,8,9,3]
Pivot 2 is now at index 1 - its correct final position. Return 1.
```

Tracing partition on `[8, 3, 5, 1, 9, 2]` with pivot 2: scanning finds only one element, 1, smaller than the pivot, at `j=3`. This triggers `i` advancing to 0 and a swap that moves 1 to the front. The final swap places the pivot (2) at index `i+1 = 1`, giving `[1, 2, 5, 8, 9, 3]`. Checking the result: everything before index 1 (just the value 1) is smaller than 2, and everything after index 1 (5, 8, 9, 3) is larger than 2 — the pivot has genuinely reached its correct final sorted position in a single pass.

Quicksort itself, in code

```
def quicksort(arr, low=0, high=None):
    if high is None:
        high = len(arr) - 1
    if low < high:
        pivot_index = partition(arr, low, high)
        quicksort(arr, low, pivot_index - 1)     # recurse on the smaller region
        quicksort(arr, pivot_index + 1, high)     # recurse on the larger region
    return arr
```

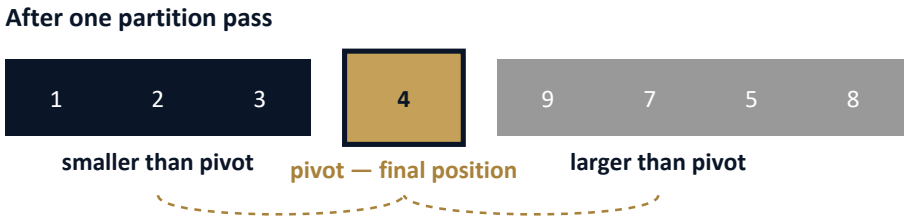
Quicksort's structure is the mirror image of merge sort's: partition does the hard rearranging work *first*, immediately placing one element (the pivot) in its final position, and the two recursive calls handle everything else. There is no merge step at the end at all — once both the smaller-region and larger-region recursive calls finish, sorting each region internally, the entire array is already sorted, since the pivot between them is already correctly placed. This is also why quicksort sorts in place: `partition` only ever swaps elements within the existing array, never allocating a second array the way merge sort's `merge` function does.

Tracing quicksort's full recursion on `[8, 3, 5, 1, 9, 2, 7, 4]`

```
quicksort([8,3,5,1,9,2,7,4], 0, 7):  pivot=4. partition → [3,1,2,8,9,7,5,4]→ swap pivot to i
  → [3,1,2,4,9,7,5,8], pivot_index=3
  quicksort(low=0,high=2) on [3,1,2]:  pivot=2 → [1,2,3], pivot_index=1
    quicksort(0,0): base case
    quicksort(2,2): base case
  quicksort(low=4,high=7) on [9,7,5,8]: pivot=8 → [7,5,8,9], pivot_index=6... (continues recu
Final result: [1,2,3,4,5,7,8,9]
```

Tracing the full recursion on this 8-element input: the first partition places 4 at index 3, splitting the array into `[3,1,2]` (indices 0-2, all smaller than 4) and `[9,7,5,8]` (indices 4-7, all larger than 4). Each of these regions is then independently partitioned and recursed upon, continuing until every region shrinks to a base case of size 0 or 1. The key structural point, worth comparing directly against merge sort's trace from Lecture 1, is that each pivot lands in its correct position *immediately*, before either recursive call even begins — there is no final combining step required afterward.

The picture: partition splitting around a placed pivot



Recurse independently on each group — the pivot itself is never touched again.

This single image captures quicksort's core mechanism: after one partition pass, the pivot sits exactly at the boundary between two groups, in its true final position — the recursive calls that follow only need to sort within each group, never touching the pivot again.

Average case: $\Theta(n \log n)$, when partitions are balanced

When each partition happens to split its range roughly in half — which occurs on average across random inputs and random pivot positions — the resulting recursion tree has $\log_2 n$ levels, structurally identical in shape to merge sort's recursion tree from Lecture 1. Each level's total partitioning work is O(n), since partition makes one pass over its current range. Multiplying gives $\Theta(n \log n)$ total time — matching merge sort's guarantee, but critically, only as an *average*-case result here, not a guarantee for every possible input, which the next slide examines.

Worst case: $\Theta(n^2)$, when partitions are maximally unbalanced

If the chosen pivot happens to always be the smallest or largest element remaining in its range — which occurs, for example, on an already-sorted input when the last element is always chosen as the pivot — one side of every partition is completely empty. The recursion tree degenerates into a single long chain of n levels rather than a balanced tree of $\log_2 n$ levels. Each level still costs O(n) for the partition scan, giving a total of $O(n) \times O(n) = \Theta(n^2)$ — the same order as Week 5's simple sorts, and dramatically worse than the average case just established.

A second worked partition trace, with a middling pivot

A second partition trace, on `[4, 1, 7, 3, 9, 2]` with pivot 2, mirrors the first trace's structure closely, since 2 is again a small value relative to most of the array — only 1 is smaller. This produces a similarly unbalanced split, with just one element (1) in the smaller region and four elements (7, 3,

9, 4) in the larger region. This is worth noticing: whether a given partition is balanced or unbalanced depends entirely on where the pivot's value happens to rank among the current range, not on any property of the partition algorithm itself.

Fixing the worst case: pivot selection strategies

Choosing the pivot randomly, rather than always the last element, makes the specific already-sorted-input worst case (and similar adversarial patterns) extremely unlikely to be triggered reliably, since the probability of repeatedly drawing the worst possible pivot at every level becomes vanishingly small. A common practical compromise, median-of-three, picks the median value among the first, middle, and last elements of the current range as the pivot, which resists many common problematic input patterns cheaply. It is important to be precise: neither technique *eliminates* the $\Theta(n^2)$ theoretical worst case entirely — a sufficiently adversarial or unlucky input can still trigger it — but both make it practically negligible, which is a genuinely different and weaker guarantee than merge sort's unconditional $\Theta(n \log n)$.

Space cost: quicksort vs. merge sort

Placing both advanced sorts side by side: merge sort guarantees $\Theta(n \log n)$ in every case at the cost of O(n) extra space; quicksort matches this only on average, degrading to $\Theta(n^2)$ in the worst case, but needs only O(log n) extra space for its recursion stack (when partitions are reasonably balanced), since it sorts in place with no merge step. Quicksort as commonly implemented is also not stable, since the partition swaps can reorder equal elements arbitrarily. Neither algorithm dominates the other outright — the choice again depends on which cost a specific application can better afford, exactly the same decision-making theme running throughout this course.

MTech addition — the recurrence for the worst case

The worst-case recurrence is $T(n) = T(n-1) + O(n)$ — only one recursive call, since one side of the partition is empty, on $n-1$ remaining elements. Expanding this recurrence gives $O(n) + O(n-1) + O(n-2) + \cdots + O(1) = O(n^2)$, exactly the same arithmetic series structure used throughout this course since Week 2. Comparing this directly to the balanced-case recurrence, $T(n) = 2T(n/2) + O(n) \Rightarrow \Theta(n \log n)$, the entire difference between quicksort's best and worst behavior comes down to whether each partition step is recursively followed by one call on nearly the full remaining input, or two calls on roughly half each — the partition step itself costs $O(n)$ either way.

Exercise

As an exercise, trace the `partition` function by hand on the already- sorted input `[1, 2, 3, 4, 5]` using the last-element pivot rule, and confirm it produces a maximally unbalanced split at every level — one empty side each time — matching this lecture's description of the worst case exactly. Then implement a randomized-pivot version of `partition` and empirically compare its actual recursion depth against the deterministic last-element version on a 1,000-element already-sorted input, confirming the randomized version avoids the worst-case chain in practice.

Real-world use of quicksort

Quicksort's excellent average-case performance and in-place memory behavior make it the practical default in many systems — C's standard library `qsort` and the C++ Standard Library's `std::sort` historically use quicksort or a closely related hybrid. Many modern implementations use introsort, which runs ordinary quicksort but monitors recursion depth, and switches to heapsort — covered in Week 8, with a guaranteed $O(n \log n)$ worst case — if the depth suggests the pathological worst-case pattern is occurring. This hybrid captures quicksort's typical speed advantage while still guaranteeing merge-sort-like worst-case behavior, combining the strengths of both algorithms covered this week.

Quiz — predict the outcome

Before checking the answer, determine whether running quicksort with the last-element pivot rule on the already-sorted array `[1, 2, 3, 4, 5, 6]` produces the best case or the worst case, and explain precisely why, using this lecture's partitioning mechanics.

Quiz — answer

This is the worst case. On an already-sorted array, the last element in any range is always the largest element remaining in that range, so every partition places the pivot at the very end of its own range, leaving the "larger" side completely empty and the "smaller" side containing every other remaining element. The recursion tree degenerates into a chain of 6 levels rather than the balanced $\log_2 6 \approx 2.6$ levels the average case would produce, giving $\Theta(n^2)$ total work rather than $\Theta(n \log n)$ — exactly the failure mode pivot-selection strategies are designed to avoid.

A second quiz check — random pivot

As a follow-up, consider sorting the same already-sorted array `[1, 2, 3, 4, 5, 6]`, but using a randomized pivot rule instead of always choosing the last element. Before checking the answer, determine whether the worst case is still guaranteed to occur.

Second quiz — answer

No, the worst case is not guaranteed with a randomized pivot rule. A randomly chosen pivot is unlikely to happen to be the extreme value at every single level of the recursion, so the maximally unbalanced chain this lecture traced for the deterministic last-element rule is no longer the expected outcome. It remains theoretically *possible* — an extraordinarily unlucky sequence of random choices could still reproduce it — but it is no longer *guaranteed* by this specific already-sorted input, which is the precise and important distinction between genuinely eliminating a worst case and merely making it statistically improbable, introduced earlier in this lecture.

Summary

Quicksort partitions its input first, immediately placing one element in its final sorted position, then recurses independently on the two resulting regions, with no merge step required afterward. Its average-case time is $\Theta(n \log n)$, matching merge sort, but only when partitions happen to be reasonably balanced; its worst case is $\Theta(n^2)$, arising from maximally unbalanced partitions, which a naive pivot rule can trigger reliably on already-sorted input. Quicksort sorts in place, using only O(log n) extra space for its recursion stack, cheaper than merge sort's O(n), but without merge sort's unconditional worst-case guarantee. Next lecture consolidates every sorting algorithm covered across Weeks 5 and 6 into a single decision framework for choosing the right one for a real workload.